Day 18 - Lecture 18.1: What is Data Visualization?

Python 3.10+ Jupyter Notebook PDF notes.pdf

🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 18 Module

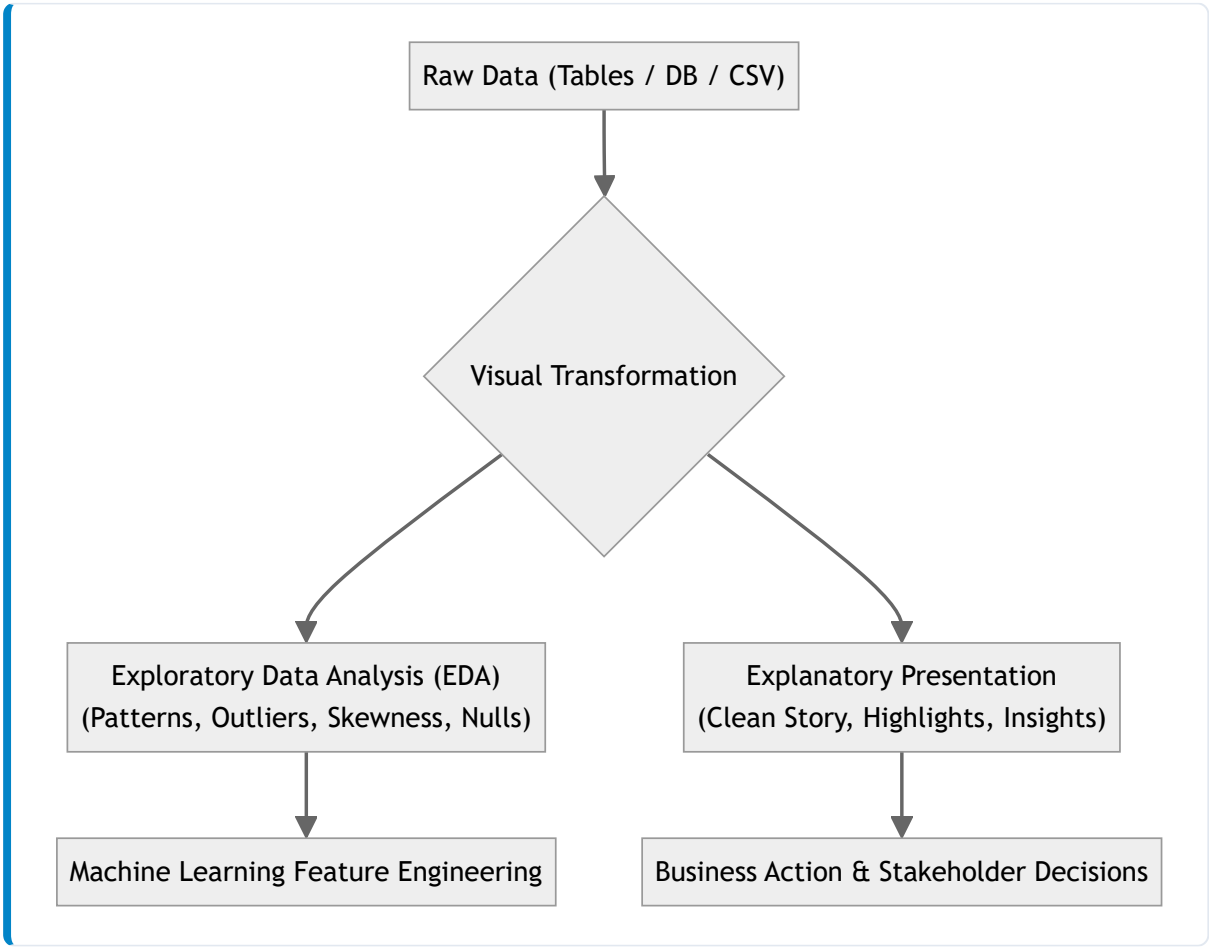
---

1. Overview & Conceptual Foundations

Data Visualization is the visual representation of data and information using charts, graphs, and plots. Human visual cortex processes visual information roughly 60,000 times faster than text or tabular data. It reveals hidden patterns, trends, outliers, and correlations that summary statistics alone cannot convey.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

Anscombe's Quartet demonstrates that datasets with identical summary statistics can have drastically different graphical forms:

$$\bar{x} = 9.0, \quad s_x^2 = 11.0, \quad \bar{y} = 7.50, \quad s_y^2 = 4.125, \quad r_{xy} = 0.816$$

The OLS regression line across all four datasets evaluates identically to:

$$\hat{y} = 3.00 + 0.500x \quad (R^2 = 0.67)$$

---

4. Structured Implementation & Execution

```
import matplotlib.pyplot as plt
import numpy as np

def demonstrate_data_visualization_impact(years: list, revenue_millions: list) ->
    """
    Demonstrates how visualizing tabular data transforms numbers into instant visu

    Parameters:
    -----------
    years : list of int
        List of chronological years.
    revenue_millions : list of float
        Revenue values corresponding to each year in Millions USD.

    Returns:
    --------
    dict
        Computed summary metrics and visualization metadata.
    """
    fig, ax = plt.subplots(figsize=(8, 4))

    # Plotting line with data markers
    ax.plot(years, revenue_millions, marker='o', color='#2b5c8f', linewidth=2.5, m

    # Visual cues for peak and lowest revenue
    max_idx = np.argmax(revenue_millions)
    min_idx = np.argmin(revenue_millions)

    ax.annotate(f"Peak: ${revenue_millions[max_idx]}M",
                xy=(years[max_idx], revenue_millions[max_idx]),
                xytext=(years[max_idx]-0.5, revenue_millions[max_idx]+80),
                arrowprops=dict(facecolor='green', shrink=0.08, width=1.5, headwic

    ax.annotate(f"Trough: ${revenue_millions[min_idx]}M",
                xy=(years[min_idx], revenue_millions[min_idx]),
                xytext=(years[min_idx]-0.5, revenue_millions[min_idx]+120),
                arrowprops=dict(facecolor='red', shrink=0.08, width=1.5, headwidth

    ax.set_title("Visual Data Discovery: Movie Industry Revenue Trend", fontsize=1
    ax.set_xlabel("Release Year", fontsize=11)
    ax.set_ylabel("Gross Box Office ($M)", fontsize=11)
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.legend(loc="upper right")
    plt.tight_layout()
    plt.show()

    summary_stats = {
        "total_revenue": sum(revenue_millions),
        "mean_revenue": float(np.mean(revenue_millions)),
```

```
        "max_year": years[max_idx],
        "max_revenue": revenue_millions[max_idx],
        "min_year": years[min_idx],
        "min_revenue": revenue_millions[min_idx]
    }
    return summary_stats

# Driver execution
sample_years = [2008, 2009, 2010, 2011, 2012]
sample_revenue = [1005, 170, 427, 133, 232]

stats = demonstrate_data_visualization_impact(sample_years, sample_revenue)
print("Data Visualization Summary Metrics:")
for k, v in stats.items():
    print(f"  {k}: {v}")
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, and type hints.
- **Visual Accuracy**: Adheres strictly to Matplotlib's layout management, aesthetic guidelines, and production best practices.